
BREAKING THE CONTEXT BOTTLENECK ON LONG TIME SERIES FORECASTING *

Chao Ma, Yikai Hou, Xiang Li, Yinggang Sun, Haining Yu, Zhou Fang, Jiaxing Qu

ABSTRACT

Long-term time-series forecasting is essential for planning and decision-making in economics, energy, and transportation, where long foresight is required. To obtain such long foresight, models must be both efficient and effective in processing long sequence. Recent advancements have enhanced the efficiency of these models; however, the challenge of effectively leveraging longer sequences persists. This is primarily due to the tendency of these models to overfit when presented with extended inputs, necessitating the use of shorter input lengths to maintain tolerable error margins. In this work, we investigate the multiscale modeling method and propose the Logsparse Decomposable Multiscaling (LDM) framework for the efficient and effective processing of long sequences. We demonstrate that by decoupling patterns at different scales in time series, we can enhance predictability by reducing non-stationarity, improve efficiency through a compact long input representation, and simplify the architecture by providing clear task assignments. Experimental results demonstrate that LDM not only outperforms all baselines in long-term forecasting benchmarks, but also reducing both training time and memory costs.

Keywords Time Series Forecasting · Multiscale Analysis · Multiscaling · Neural Network

1 Introduction

Time series analyses are widely used across various industries, including power load forecasting, anomaly detection in credit fraud, imputation in environmental monitoring, and user classification based on their behavior history. Among these tasks, time series forecasting is an important analytical technique that uses historical sequences to predict future outcomes. Based on the length of the forecast horizon, time series forecasting can be divided into short-term and long-term categories. Because time series are nearly stationary in short-term, i.e, the mean and variance of the data do not vary much over time, short-term forecasting is usually easier. Long-term forecasting, on the other hand, is much more complicated due to various non-stationary factors, especially trends. Certain planning tasks, including infrastructure development, climate change adaptation, urban planning, resource management, and supply chain optimization, demand the foresight that only long-term forecasting can provide. As a result, long-term forecasting proves to be more broadly applicable across diverse fields, such as *economics and finance*[1, 2, 3, 4], *energy*[5, 6, 7], *transportation*[8, 9, 10], *environment*[11, 12], and *industry*[13, 14, 15].

Long-term Time-Series Forecasting (LTSF) relies on effective and efficient handling of long contexts and modeling of long-term dependencies. Recent advancements have led to the development of network architectures specifically designed to exploit the intrinsic properties of time series data, such as *periodicity*[16, 17, 18], *multiscale*[19, 20], *segmented*[21, 22], and *non-stationary*[23, 24, 25]. Among these methods, the multiscale modeling approach[26], as exemplified by TimeMixer[19], has demonstrated significant promise in long-term dependencies modeling. This is particularly due to its close alignment with real-world phenomena such as traffic, which exhibits patterns at multiple temporal levels, including daily and hourly variations. The TimeMixer model leverages temporal patterns across different sampling scales with two main modules: the Past-Decomposable-Mixing (PDM) and the Future-Multipredictor-Mixing (FMM). The PDM module independently mixes seasonal and trend components at different scales. The FMM module integrates multiple forecasters to exploit complementary capabilities in multi-scale observations for improved accuracy. However, the sampling approach exhibits three key limitations that limit its range of application:

*contact hoyikai@outlook.com or machao@hrbust.edu.cn for any question. You can also open an issue in github.

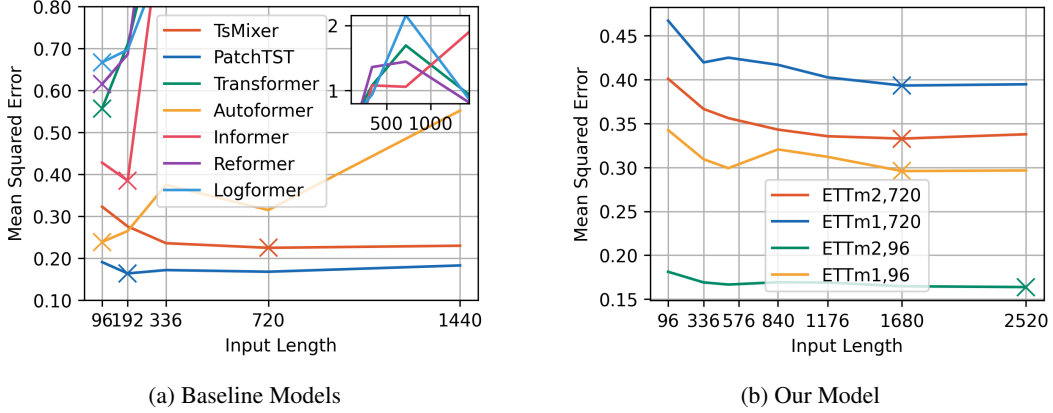


Figure 1: Prediction error vs. input length. (a) represents the performance of different baseline models in the 96-step prediction setting on the ETTm2 dataset. (b) shows the performance of our model under different experiment settings, demonstrating its ability to handle longer inputs.

1. **Insufficient context.** Prediction error for current methods is only acceptable when using shorter inputs. The trends provided by multiscale downsampling cover only a short time period, resulting in a lack of sufficient context for the model to learn and generalize effectively.
2. **None-stationarity.** When the input sequence is non-stationary, the downsampling process subsequently introduces additional non-stationary components across multiple scales, thereby increasing complexity and creating additional challenges for the model.
3. **Not generally applicable.** Multiscale downsampling components require specially designed communication or aggregation modules to be aggregated together (like PDM). This introduces significant additional overhead, which compromises generality and scalability.

The issue of short input sequences was identified early[27] and was particularly acute in early Transformer methods, such as Informer[28], Autoformer[16], and FEDformer[17], as illustrated in Fig.(1a). Overfitting to noise in the input sequence is a major cause. Previous works[17, 29] employed Fourier analysis to reduce overfitting by randomly selecting frequency components, which lowers computational complexity but does not fully solve the problem. Recent methods, such as PatchTST[22], Crossformer[21], and iTransformer[30], expand the reception field to address this issue, but risk losing local details and increasing memory costs. Regarding the non-stationary issue, methods such as RevIN[25] and the Non-Stationary Transformer[24] have introduced two-stage normalization approaches that partially alleviate the problem. Nevertheless, managing non-stationary signals remains a significant challenge for neural networks. General applicability is a highly desirable trait. However, downsampling-based multiscale methods exhibit task overlap between different scales when handling low-frequency information, leading to conflicts during final aggregation. There are two solutions: one uses a serial structure in which each layer handles a different scale, as seen in N-HITS[20], Multiscale ViT[31], and MeMVIT[32]. The other introduces modules for cross-scale communication, such as TimeMixer[19], HRViT[33], and CrossViT[34]. Both methods impose constraints on the predictor, reducing general applicability and incurring additional overhead. These limitations are prevalent in current LTSF methods and continue to affect existing multiscale analysis approaches. Our proposed Logsparse Decomposable Multiscaling (LDM) framework addresses these limitations while achieving both high performance and efficiency.

To the best of our knowledge, this is the first work to investigate multiscale decomposition methods in LTSF. In our experiments, LDM achieves state-of-the-art results on all long time-series forecasting benchmarks, while significantly reducing both training time and memory usage. Our contributions can be summarized as follows:

- We introduce the *Logsparse Scale*, which, for the first time, addresses overfitting issues associated with long sequences in multiscale scenarios (see Fig.1b). A longer context enables the model to capture multi-scale features more effectively.
- We present a straightforward multiscale decomposition method for arbitrary-scale multi-resolution analysis, effectively addressing the non-stationarity problem inherent in current downsampling approaches.
- We propose a generally applicable framework, Logsparse Decomposable Multiscaling (LDM), which significantly enhances the performance of existing forecasting approaches while reducing training time and memory overhead.

2 Related Work

2.1 Time Series Forecasting

Current mainstream methods, mostly based on MLP and Transformer, have addressed the issue of error accumulation inherent in autoregressive prediction approaches, particularly prevalent in recurrent neural networks[35, 36]. Early Transformer variants primarily focused on reducing attention complexity, exemplified by approaches like Autoformer[16] and FEDformer[17], Informer[28] which introduced Fourier analysis and sequence decomposition methods into time series forecasting. The latest Transformer method, PatchTST[22], Crossformer[21] adopts strategies such as channel-wise independence and Patch embedding methods, which enhanced temporal dependencies modeling abilities, effectively enhancing prediction accuracy. MLP-based techniques often exploit inductive biases from time series analysis, as exemplified by N-BEATS[37] and N-HITS[20], which employ boosting techniques. Non-stationary Transformer[24] and RevIN[25] focus on analyzing non-stationarity, TimesNet[18] leverages multi-periodicity, TSMixer[38] and iTransformer[30] captures multivariate correlations, and TimeMixer[19] exploits multiscale features. However, the optimal input size for these methods is often much shorter than the prediction length, creating a significant bottleneck for LTSF[39].

This raises the question: *What determines the input size of neural networks for time series forecasting?* Through a comparative analysis of mainstream methods, we observe a positive correlation between token size and input length. Token size, referring to the model’s smallest processing unit (or receptive field), determines how much input a model can handle. For instance, Autoformer[16] and FEDformer[17], which treat individual time points as tokens, perform best with a context window of 96. In contrast, models like PatchTST[22] and Crossformer[21], using tokens of 8 to 16 time points, extend the window to 336. MLP-based models, such as TSMixer[38], N-BEATS[37], and N-HITS[20], handle up to 512 due to their ability to process entire sequences as tokens. *Token size also significantly impacts model behavior.* Models with smaller tokens[40, 41, 28, 17, 22] excel at capturing local variations like subtle periodicities, while those with larger tokens emphasize global patterns, such as mean and variance[37, 20, 38]. Based on these observations, we propose: *Token size governs the input length a model can accommodate.* However, larger tokens risk missing local variations. A model that processes tokens of varying sizes simultaneously, as in Multiscale Modeling[26], may provide the optimal solution.

2.2 Multiscale Modeling

Multiscale Modelling is widely used in Computer Vision [42, 43, 44, 45, 46], e.g. in Multi-view stereo (MVS) [47, 48, 49, 50] and Dense prediction[51, 52] rely on multi-scale features to improve prediction accuracy, while efficiency can be improved in video and image recognition[31, 53, 54]. In the field of time-series forecasting, recent multiscale research efforts include N-HITS[20] and TimeMixer[19]. In terms of multiscale modelling, N-HITS belongs to a serial structure, stacking multiple layers of MLPs, with early layers using lower temporal resolution, and the final predictions are generated by aggregating the predictions produced by each layer. TimeMixer belongs to a parallel structure, where each scale is modelled independently by a multilayered feed-forward neural network as well as a linear predictor, and features of each scale is aggregated by the proposed decomposable multiscale mixing layer. However, the overhead of the mixing layers is substantial. Our approach leverages the efficiency of parallel structures in multiscale modeling while eliminating the need for a mixing layer, achieving both high performance and computational frugality.

3 Methodologies

Preliminaries. LTSF can be modeled as a sequence-to-sequence (seq2seq) supervised learning task[55]. For a multivariate dataset, starting from an early time point L to a near end point $T - H$, sample a portion of historical data $\mathbf{x}_t$ and future data $\mathbf{y}_t$ at each time point t to form a sequence pair $\{\mathbf{x}_t, \mathbf{y}_t\}_{t=L}^{T-H}$, where T is the length of the dataset. Given a multivariate historical sequence $\mathbf{x}_t \in \mathbb{R}^{M \times L}$ at time t , where L is the context window size (or input sequence length), the objective of LTSF is to predict its future sequence $\mathbf{y}_t \in \mathbb{R}^{M \times H}$, where H is the prediction length. The multivariate time series comprises multiple dimensions, where each dimension $i \in \{1, 2, \dots, M\}$ represents a separate time series $x^{(i)} \in \mathbb{R}^L$, referred to as a channel. Our framework handles multiple channels simultaneously but ignores potential correlations between channels, as cross-dimensional dependencies[38, 21] are beyond the scope of this study. We formalize the LTSF problem f as follows:

$$f : \left\{ \mathbf{x}_t^{(i)} \right\}_{i=1}^M \longrightarrow \left\{ \hat{\mathbf{y}}_t^{(i)} \right\}_{i=1}^M \quad (1)$$

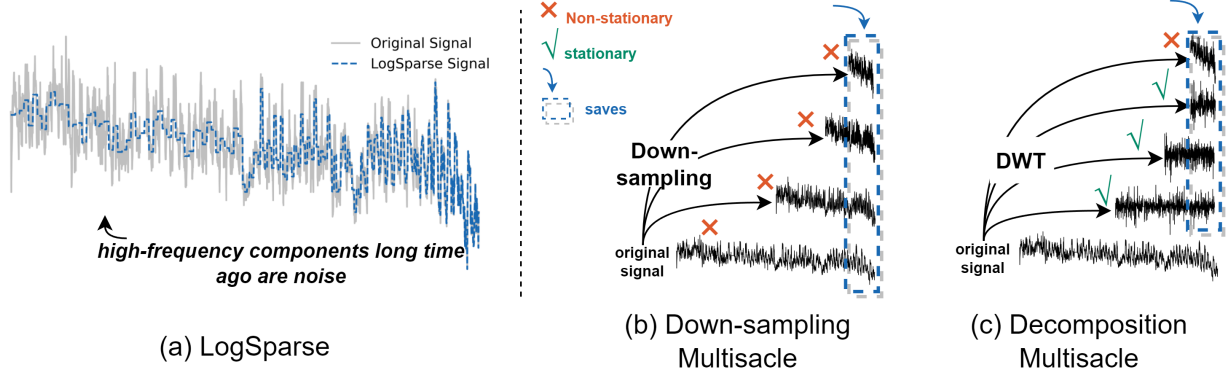


Figure 2: **Motivations.** (a) shows the idea of LogSparse Scale, where the details of temporal variations diminishes as time recedes; (b) shows current Down-sampling Multiscale approach from TimeMixer[19]; and (c) shows Decomposition Multiscale approach. The primary distinction lies in the stationarity of each scale. To implement Logsparse Scale, we retain only the tail of each scale, as indicated by the blue box in the figure.

The training objective is to minimize the loss $\mathcal{L}$ represents the error between the predicted values $\hat{\mathbf{y}}_t^{(i)}$ and the true values $\mathbf{y}_t^{(i)}$, such as Mean Squared Error (MSE):

$$\mathcal{L}(\mathbf{y}_t^{(i)}, \hat{\mathbf{y}}_t^{(i)}) = \frac{1}{H} \sum_{i=1}^H (\mathbf{y}_t^{(i)} - \hat{\mathbf{y}}_t^{(i)})^2 \quad (2)$$

3.1 Logsparse Scale

According to Dow Theory[56], the stock market's price trends are categorized into three levels of duration and granularity: Primary, Secondary, and Minor, with an emphasis on long-term trends and recent fluctuations. This theory posits that daily fluctuations from a month ago are generally not considered significant for current forecasting. Drawing inspiration from this, we propose the *Logsparse scale*, designed to mitigate the issue of input overfitting, which leads to **Insufficient context**.

The concept of Logsparse was introduced in LogTrans[57] as a variant of the attention mechanism, in which the density of attention computations grows increasingly sparser on a logarithmic scale as the time interval increases. Logsparse Scale adopts this principle, focusing selectively on high-frequency components over time, as shown in Fig.2(a). Implementing this concept is straightforward: it involves adding a new truncation step, named Logsparse truncation, at the end of the current downsampling approach, as shown in Fig.2(b). The concept of Logsparse truncation involves selectively reducing the length of sequences based on their scale, with a focus on diminishing the length of high-frequency, small-scale sequences more significantly than that of large-scale sequences. This method is particularly useful in contexts where it is beneficial to compress data while preserving the most significant, large-scale information.

Sparsity Parameter η . This new parameter η , which ranges between 0 and 1 (inclusive of 1 but exclusive of 0), quantifies the degree of sparsity. The closer η is to 1, the less aggressive the truncation, meaning sequences are kept longer. Conversely, a smaller η value leads to more substantial truncation, effectively "diluting" the sequence by reducing its length more significantly.

The formula for the post-truncation length $\tilde{L}_n$ of a component at scale n is given by:

$$\tilde{L}_n = \min \left(\frac{p^n}{\eta}, L_n \right) \quad (3)$$

where η adjusts the scale factor inversely with the sparsity degree, meaning that a smaller η (indicating higher sparsity) results in a smaller limit for $\tilde{L}_n$, thus enforcing a more significant reduction in sequence length.

3.2 Multiscale Decomposition

The problem of **None-stationarity** can be mitigated by using decomposition methods. For example, the Discrete Wavelet Transform (DWT)[58] is also a multiscale method that decomposes the time series into one non-stationary

approximate coefficient and a set of stationary detailed coefficients, as shown in Fig.2(c). We first introduce the concept of MultiResolution Analysis (MRA), and then present the method we have proposed.

In observing objects, the human eye features a wide field of view with low resolution at long distances, capturing only basic outlines. Up close, the field narrows but resolution improves, allowing for detailed views. MultiResolution Analysis, or Multiscale Analysis, applies this concept by representing a function $f(t)$ through successive, smoothed approximations at varying resolutions. This technique, common in computer vision[42, 43, 44, 45, 46], involves decomposing an image at multiple scales to compare or aggregate the data for enhanced information extraction.

Wavelet analysis is among the most widely used MRA methods. The continuous wavelet transform equation of the signal $s(t)$ is as follows:

$$W(a, b) = \frac{1}{\sqrt{a}} \int \bar{\psi} \left(\frac{t-b}{a} \right) s(t) dt \quad (4)$$

where a represents a time dilation or scale, and b represents a time translation, which in practice usually takes the discrete value 2^i , to obtain the following Discrete Wavelet Transform (DWT)[58]:

$$w(2^i, 2^i n) := \frac{1}{\sqrt{2^i}} \sum_k \bar{\psi} \left(\frac{k}{2^i} - n \right) s(k) \quad (5)$$

The performance of the wavelet transform is significantly affected by the chosen wavelet basis ψ , and identifying suitable wavelet bases for different tasks is challenging. In addition, the time dilation $a = 2^i$ in the discrete wavelet transform is predefined and does not permit the use of arbitrary values, implying that one can only observe the signal at prefixed scales, whereas in reality, specific scales are of greater interest for particular datasets. Fig. 3 depicts an implementation of the discrete wavelet transform, known as the Filter Bank Structure. g and f are filters that depend on ψ , and the down-arrow represents downsampling with scale $Q = 2$. To address the problems associated with the wavelet transform, we implemented two modifications to the existing architecture.

Filters g and f . In the traditional wavelet transform, g and f are related to the wavelet function ψ . In this modified approach: (1) f is replaced by a simple moving average filter, which is a type of low-pass filter. Low-pass filters allow low-frequency components to pass through while attenuating the higher frequency components. The moving average filter achieves this by averaging the data points within a specified window of size p , smoothing out rapid fluctuations and noise. (2) $g = 1 - f$ implies that g acts as a high-pass filter, capturing the residual signal that is not accounted for by f . This helps in retaining the detail or high-frequency components of the signal.

Downsampling $\downarrow_Q$. Initially, the downsampling scale Q was 2, halving the signal at each filter bank level. We've revised this to $Q = p/2$, where p is the window size of the moving average filter. This adjustment ensures the sampling rate exceeds the Nyquist rate, preventing aliasing and information loss. By setting $Q = p/2$, our approach reduces redundancy and noise, enhances computational efficiency, and maintains data integrity.

After n rounds of filtering, the original signal is decomposed into a set of relatively smooth high-frequency signals $\{w^i\}$, and a none-stationary low-frequency signal s^n , which is calculated as follows:

$$\begin{aligned} \tau^i(k) &= \sum_{n=-p^i/2}^{p^i/2} \frac{1}{p^i} \cdot s^i(k-n) \\ s^{i+1}(k) &= \downarrow_{\frac{p^i}{2}} \tau^i(k), \quad w^i(k) = s^i(k) - \tau^i(k) \end{aligned} \quad (6)$$

where τ^i is the low-pass filter result, $s^0 = s$ is the original signal, s^i is the low-pass component at the i level, and w^i is the high-pass component at the i level. The $\downarrow_{\frac{p^i}{2}}$ is the downsampling, which is implemented by avgpool of $kernel\ size = stride = \frac{p^i}{2}$.

Scale Factor $\{p^n\}$. The method introduces a set of window size parameters $\{p^1 < p^2 < \dots < p^n\}$, which define different scales of analysis without relying on traditional wavelet bases. Each p^n acts as a *scale factor* for its respective scale n , adjusting the level of detail and smoothing applied to the signal at each stage.

3.3 Logsparse Decomposable Multiscaling Framework

We propose the **Generally applicable** framework named Logsparse Decomposable Multiscaling(LDM). LDM does not rely on any aggregation or communication modules, nor does it make any assumptions about the predictor.

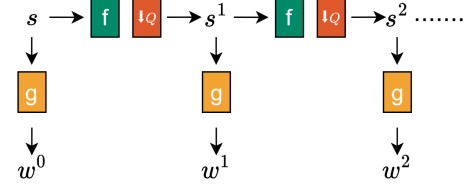


Figure 3: A wavelet filter bank structure is described in [58]. The down-arrow symbolizes downsampling; when $Q = 2$, the downsampling operation halves the original length. The symbols f and g represent a low-pass and a bandpass filters, respectively.

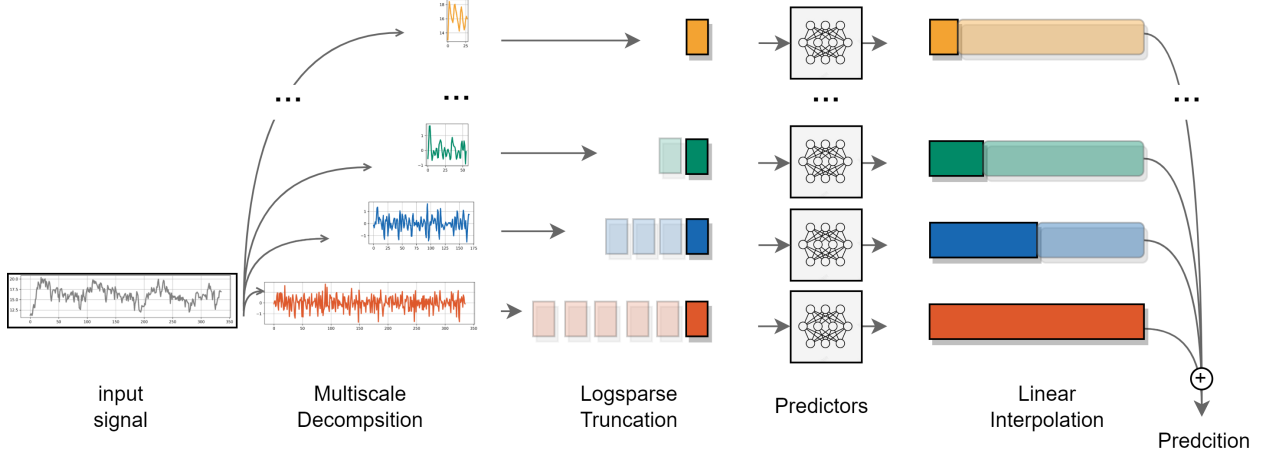


Figure 4: Overall Framework.

The overall structure of the model is shown in Fig.4. Specifying a set of scales $\{p^1, p^2, \dots, p^N\}$, the input sequence x undergoes multi-scale decomposition to obtain sequences of different scales. The sequences, after Log-Sparse Truncation, are then fed into each predictor for prediction. Finally, the resulting predictions for each scale are interpolated to the same length and aggregated as the final prediction.

Technically, given a signal s , a multiscale decomposition is applied to obtain a set of multiscale time series $S = \{s_1, s_2, \dots, s_{N+1}\}$. Series s_n at scale n possesses dimensions $s_n \in \mathbb{R}^{M \times L_n}$, where L_n , its length, is calculated as $L_n = 2 \frac{L}{p^n}$. Here, L represents the length of the original input, and p^n is the scale factor. For each scale n , a corresponding prediction length $H_n = 2 \frac{H}{p^n}$ is defined, where H is the original prediction length. Subsequently, each series undergoes truncation using LogSparse Truncation with a sparsity parameter η , resulting in a new length $\tilde{L}_n$. Utilizing $\{\tilde{L}_n\}_{n=1}^{N+1}$ and $\{H_n\}_{n=1}^{N+1}$, $N+1$ predictors are designated, each predicting a sequence $y_j \in \mathbb{R}^{M \times H_n}$. Finally, linear interpolation is utilized to complete these predicted sequences, resulting in the final prediction y :

$$\begin{aligned} y_n &= \text{Predictor}_n(s_n), n \in \{1, \dots, N+1\}, \\ y &= \sum_{n=0}^{N+1} \text{Interpolate}_{H_n \rightarrow H}(y_n), \end{aligned} \quad (7)$$

3.4 Transformer-based Predictor

Transformer models like PatchTST excel in LTSF due to their patch embedding layers, while FEDformer is better suited for short-term forecasts, focusing on individual time points. Our goal is to develop a unified predictor capable of handling both long and short-term forecasts, eliminating the need for separate models for different time scales. To this end, we integrate both traditional vanilla embeddings and innovative patch embedding layers to process input series separately, as illustrated in Fig. 5. This approach aims to leverage the strengths of both embedding techniques to enhance the model's forecasting capabilities across different time scales.

Given the sequence $\tilde{s}$ and the corresponding scale p , the input sequence S is unfolded into a 2D tensor as follows:

$$\tilde{s}^{2D} = \text{Unfold}_p(\text{Padding}(\tilde{s}^{1D})) \quad (8)$$

Here, $\text{Padding}(\cdot)$ adds zeros to extend the time series along the temporal dimension, ensuring compatibility with $\text{Unfold}_p(\cdot)$, where both the unfold size and stride are set to p . The resulting 2D tensor, $\tilde{s}^{2D}$, features dimensions $\mathbb{R}^{M \times c \times p}$, where each row correlates with a patch embedding $w_1 \in \mathbb{R}^{M \times c \times d_{model}}$ and each column correlates with a vanilla embedding $w_2 \in \mathbb{R}^{M \times p \times d_{model}}$. These two sets of embedding vectors undergo processing through separate Transformer encoders,

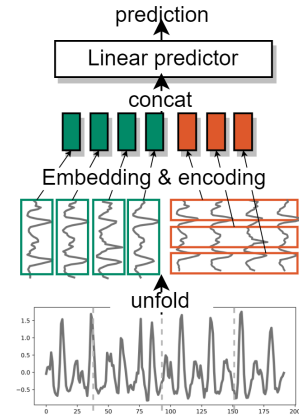


Figure 5: The proposed Transformer predictor. The unfolded input sequence is fed into two separate sets of embeddings and encoders.

following which they are concatenated and flattened into $w \in \mathbb{R}^{M \times (p+c)d_{model}}$. Finally, a linear predictor $\mathbb{R}^{(p+c)d_{model} \times H}$ is employed to generate the forecast.

4 Experiments

We first demonstrate the SoTA performance of the framework using the proposed predictor, and then use multiple models as predictors to demonstrate that the framework is generally applicable. We will then argue that such performance improvements actually stem from reduced costs, thanks to multiscale decomposition which converts a complex problem into a set of simple problems. Finally, we dissect the characteristics of the method through ablation experiments and interpretability analyses. We begin by outlining the experimental setup, which follows the established configurations of previous studies[28, 16, 17, 22, 59].

Table 1: Long-term time series forecasting benchmarks. The forecastability is calculated by one minus the entropy of Fourier decomposition of time series[60] and the results were provided by TimeMixer[19].

Datasets	Electricity	Traffic	Weather	Solar-Energy	ETTh1&ETTm2	ETTh1&ETTh2
Time-Series	321	862	21	137	7	7
Time-Points	26,304	17,544	52,696	52,560	69,680	17,420
Forecastability	0.77	0.68	0.75	0.33	0.46	0.46
Frequency	1 Hour	1 Hour	10 Minutes	10 Minutes	15 Minutes	1 Hour

Table 2: Experiment configuration of LDM. Input size 11;12 means Use 11 for predicted lengths of 96 and 192, and 12 for predicted lengths of 336 and 720. LR means learning rate. Scale set are freely specified scales, two numbers means the sequence will be decomposed into 3 parts.

dataset	layers	d_model	d_ff	n_heads	LR	batch size	scale set	dropout	input size
ETTh1	2	16	256	4	1.00E-04	128	{24, 168}	0.8	336;1680
ETTh2	2	16	128	4	1.00E-04	128	{24, 168}	0.8	512;1680
ETTm1	2	16	128	4	1.00E-04	128	{4, 96}	0.8	336;960
ETTm2	2	16	128	4	1.00E-04	128	{4, 96}	0.8	336;960
Weather	2	16	128	4	1.00E-04	128	{6, 144}	0.3	336;1440
Electricity	2	128	256	16	1.00E-04	32	{24, 168}	0.3	336;1680
Solar-Energy	2	128	256	16	1.00E-04	32	{6, 144}	0.3	336;1440
Traffic	2	128	256	16	1.00E-04	24	{24, 168}	0.3	336;1680

4.1 Protocols

Benchmarks. We evaluate the performance of our framework on eight main-stream benchmarks, including Weather, Traffic, Electricity, Solar-energy and 4 ETT datasets(ETTh1, ETTh2, ETTm1, ETTm2), which have been extensively used in previous works[28, 16, 17, 22, 59] and publicly available at [16]. Training/Validation/Test sets are zero-mean normalized with the mean and std of Training set, with the data split into proportions of 7:1:2 respectively. The Statistics of all benchmarks are gathered in Table 1.

Baselines. We selected eight popular State of The Art(SoTA) models as baselines, including FEDformer[17], Autoformer[16], Informer[28], Non-Stationary[24], TimesNet[18], TimesMixer[19], Crossformer[21] and PatchTST[22]. TimesMixer(MLP-based) is current SoTA baseline, and PatchTST is Transformer-based SoTA baseline.

Setup. We follow the experimental setup of mainstream methods [22]. The input length is set to 960 or 1680 most case, sometimes 336 for short-term forecasting, according to the multi-periodicity. And the prediction length is varied with $H = \{96, 192, 336, 720\}$. We utilize the Adam optimizer with Mean Squared Error($MSE = \frac{1}{n} \sum_{i=1}^n (\mathbf{y} - \hat{\mathbf{y}})^2$) as the loss function and evaluate using Mean Absolute Error($MAE = \frac{1}{n} \sum_{i=1}^n |\mathbf{y} - \hat{\mathbf{y}}|$) and MSE as metrics. The details of the hyperparameter settings are shown in Table 2. All experiments were conducted on a single Nvidia A40 48GB GPU. The source code is available at: <https://github.com/Houyikai/LDM-Logsparse-Decomposable-MultiScaling>.

4.2 Performance Analysis

SoTA Performance. We first evaluate the SoTA performance of the LDM framework with a previously proposed predictor3.4. Tables 3 and 4 concisely summarize the evaluation results of various methods, with baselines indicating the

Table 3: Multivariate long-term time-series forecasting results on all benchmarks.

Models		LDM		TimeMixer		iTransformer		PatchTST		TimesNet		Crossformer		FEDformer	
Metric		MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE
Weather	96	0.150	<u>0.202</u>	0.163	0.209	0.174	0.214	0.149	0.198	0.172	0.220	0.195	0.271	0.217	0.296
	192	0.194	<u>0.245</u>	0.208	0.250	0.221	0.254	0.194	0.241	0.219	0.261	0.209	0.277	0.276	0.336
	336	0.241	0.283	0.251	0.287	0.278	0.296	<u>0.245</u>	0.282	0.246	0.337	0.273	0.332	0.339	0.380
	720	0.302	0.331	0.339	0.341	0.358	0.349	<u>0.314</u>	<u>0.334</u>	0.365	0.359	0.379	0.401	0.403	0.428
Solar	96	0.167	0.240	0.189	0.259	0.203	0.237	0.189	<u>0.250</u>	0.373	0.358	0.232	0.302	0.286	0.341
	192	0.180	0.248	0.222	0.283	0.233	0.261	<u>0.212</u>	<u>0.266</u>	0.397	0.376	0.371	0.410	0.291	0.337
	336	0.186	0.253	0.231	0.292	0.248	0.273	<u>0.213</u>	<u>0.273</u>	0.420	0.380	0.495	0.515	0.354	0.416
	720	0.197	0.277	0.223	0.285	0.249	0.275	<u>0.223</u>	<u>0.274</u>	0.420	0.381	0.526	0.542	0.380	0.437
ECL	96	0.130	<u>0.229</u>	0.153	0.247	0.148	0.240	0.129	0.222	0.168	0.272	0.219	0.314	0.193	0.308
	192	0.144	<u>0.242</u>	0.166	0.256	0.162	0.253	<u>0.147</u>	0.240	0.184	0.322	0.231	0.322	0.201	0.315
	336	0.163	0.260	0.185	0.277	0.178	0.269	0.163	0.259	0.198	0.300	0.246	0.337	0.214	0.329
	720	0.197	0.290	0.225	0.310	0.225	0.317	0.197	0.290	0.220	0.320	0.280	0.363	0.246	0.355
Traffic	96	0.361	<u>0.255</u>	0.462	0.285	0.395	0.268	0.360	0.249	0.593	0.321	0.644	0.429	0.587	0.366
	192	0.380	<u>0.266</u>	0.473	0.296	0.417	0.276	0.379	0.256	0.617	0.336	0.665	0.431	0.604	0.373
	336	0.392	<u>0.272</u>	0.498	0.296	0.433	0.283	0.392	0.264	0.629	0.336	0.674	0.420	0.621	0.383
	720	0.430	<u>0.295</u>	0.506	0.313	0.467	0.302	<u>0.432</u>	0.286	0.640	0.350	0.683	0.424	0.626	0.382
ETTh1	96	0.366	0.395	0.375	0.400	0.386	0.405	0.379	0.401	0.384	0.402	0.423	0.448	0.395	0.424
	192	0.401	0.415	0.429	0.421	0.441	0.436	<u>0.413</u>	<u>0.429</u>	0.436	0.429	0.471	0.474	0.469	0.470
	336	0.418	0.426	0.484	0.458	0.487	0.458	<u>0.435</u>	<u>0.436</u>	0.638	0.469	0.570	0.546	0.530	0.499
	720	0.427	0.449	0.498	0.482	0.503	0.491	<u>0.446</u>	<u>0.464</u>	0.521	0.500	0.653	0.621	0.598	0.544
ETTh2	96	0.271	0.336	0.289	0.341	0.297	0.349	0.274	0.337	0.340	0.374	0.745	0.584	0.358	0.397
	192	0.331	0.375	0.372	0.392	0.380	0.400	<u>0.338</u>	<u>0.376</u>	0.402	0.414	0.877	0.656	0.429	0.439
	336	0.323	0.381	0.386	0.414	0.428	0.432	<u>0.363</u>	<u>0.397</u>	0.452	0.452	1.043	0.731	0.496	0.487
	720	0.376	0.419	0.412	0.434	0.427	0.445	<u>0.393</u>	<u>0.430</u>	0.462	0.468	1.104	0.763	0.463	0.474
ETTm1	96	0.288	0.348	0.320	0.357	0.334	0.368	0.293	0.346	0.338	0.375	0.404	0.426	0.379	0.419
	192	0.332	0.370	0.361	0.381	0.377	0.391	<u>0.333</u>	<u>0.370</u>	0.374	0.387	0.450	0.451	0.426	0.441
	336	0.361	0.387	0.390	0.404	0.426	0.420	<u>0.369</u>	<u>0.392</u>	0.410	0.411	0.532	0.515	0.445	0.459
	720	0.400	0.408	0.454	0.441	0.491	0.459	<u>0.416</u>	<u>0.420</u>	0.478	0.450	0.666	0.589	0.543	0.490
ETTm2	96	0.164	0.252	0.175	0.258	0.180	0.264	0.166	<u>0.256</u>	0.187	0.267	0.287	0.366	0.203	0.287
	192	0.213	0.294	0.237	0.299	0.250	0.309	<u>0.223</u>	<u>0.296</u>	0.249	0.309	0.414	0.492	0.269	0.328
	336	0.265	0.330	0.298	0.340	0.311	0.348	<u>0.274</u>	<u>0.329</u>	0.321	0.351	0.597	0.542	0.325	0.366
	720	0.330	0.371	0.391	0.396	0.412	0.407	<u>0.362</u>	<u>0.385</u>	0.408	0.403	1.730	1.042	0.421	0.415

Table 4: Univariate long-term time-series forecasting results on four datasets.

models		LDM		PatchTST		DLinear		FEDformer		Autoformer		Informer		LogTrans	
metrics		MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE
ETTh1	96	0.061	0.194	0.059	0.189	0.056	0.180	0.079	0.215	0.071	0.206	0.193	0.377	0.283	0.468
	192	0.069	0.209	0.074	0.215	0.071	0.204	0.104	0.245	0.114	0.262	0.217	0.395	0.234	0.409
	336	0.073	0.217	0.076	0.220	0.098	0.244	0.119	0.270	0.107	0.258	0.202	0.381	0.386	0.546
	720	0.075	0.220	0.087	0.236	0.189	0.359	0.142	0.299	0.126	0.283	0.183	0.355	0.475	0.629
ETTh2	96	0.131	0.284	0.131	0.284	0.131	0.279	0.128	0.271	0.153	0.306	0.213	0.373	0.217	0.379
	192	0.174	0.331	0.171	0.329	0.176	0.329	0.185	0.330	0.204	0.351	0.227	0.387	0.281	0.429
	336	0.166	0.329	0.171	0.336	0.209	0.367	0.231	0.378	0.246	0.389	0.242	0.401	0.293	0.437
	720	0.199	0.357	0.223	0.380	0.276	0.426	0.278	0.420	0.268	0.409	0.291	0.439	0.218	0.387
ETTm1	96	0.026	0.123	0.026	0.123	0.028	0.123	0.033	0.140	0.056	0.183	0.109	0.277	0.049	0.171
	192	0.39	0.151	0.040	0.151	0.045	0.156	0.058	0.186	0.081	0.216	0.151	0.310	0.157	0.317
	336	0.051	0.172	0.053	0.174	0.061	0.182	0.084	0.231	0.076	0.218	0.427	0.591	0.289	0.459
	720	0.066	0.197	0.073	0.206	0.080	0.210	0.102	0.250	0.110	0.267	0.438	0.586	0.430	0.579
ETTm2	96	0.064	0.191	0.065	0.187	0.063	0.183	0.067	0.198	0.065	0.189	0.088	0.225	0.075	0.208
	192	0.092	0.235	0.093	0.231	0.092	0.227	0.102	0.245	0.118	0.256	0.132	0.283	0.129	0.275
	336	0.118	0.272	0.121	0.266	0.119	0.261	0.130	0.279	0.154	0.305	0.180	0.336	0.154	0.302
	720	0.155	0.316	0.172	0.322	0.175	0.320	0.178	0.325	0.182	0.335	0.300	0.435	0.160	0.321

best results from the respective papers. LDM varies its input size from 960 to 1680, and occasionally to 336, depending on the sequence’s periodicity. PatchTST and Crossformer both utilize an input size of 336 for patch embedding. DLinear operates with an input size of 512, while other models, significantly affected by the short input problem, utilize a smaller input size of 96.

In multivariate scenarios, each model predict all variables of a multivariate time series at once. From Table 3, LDM shows best results in most cases, with an improvement of 11.7% compared to the MLP baseline TimeMixer and approximately 3.4% compared to the Transformer baseline PatchTST, in MSE. LDM performs particularly well in long-term forecasting scenarios. For example, when the prediction amplitude is 336 or above, the average improvement compared to MLP baseline is 15.3%, and the average improvement compared to Transformer baseline is 4.6%, and it shows an upward trend. Univariate results also support this observation where model predict only target dimension.

Table 5: Performance promotion obtained by our framework. original refers to the performance of the baseline model, with results obtained from FEDformer[17]. +LDM represents the results when the model is used as the predictor in our framework. Both experiments were conducted using the same experimental settings.

Methods		FEDformer (2022)		Autoformer (2022)		Informer (2021)		LogTrans (2019)		Reformer (2019)	
Metric		MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE
ETTh1	original	96	0.079	0.215	0.071	0.206	0.193	0.377	0.283	0.468	0.532
		192	0.104	0.245	0.114	0.262	0.217	0.395	0.234	0.409	0.568
		336	0.119	0.270	0.107	0.258	0.202	0.381	0.386	0.546	0.589
		720	0.142	0.299	0.126	0.283	0.183	0.355	0.475	0.628	0.666
	+LDM	96	0.062	0.191	0.062	0.191	0.063	0.194	0.058	0.185	0.059
		192	0.075	0.213	0.079	0.216	0.089	0.232	0.079	0.218	0.081
		336	0.092	0.242	0.094	0.243	0.101	0.252	0.090	0.239	0.093
		720	0.091	0.239	0.096	0.245	0.102	0.252	0.092	0.241	0.099
	promotion		26.948%	13.703%	19.984%	11.085%	55.167%	38.192%	75.834%	56.289%	86.783%
			62.620%								
ETTh2	original	96	0.128	0.271	0.153	0.306	0.213	0.373	0.217	0.379	1.411
		192	0.185	0.330	0.204	0.351	0.227	0.387	0.281	0.429	5.658
		336	0.231	0.378	0.246	0.389	0.242	0.401	0.293	0.437	4.777
		720	0.278	0.420	0.268	0.409	0.291	0.439	0.218	0.387	2.042
	+LDM	96	0.135	0.287	0.131	0.282	0.135	0.285	0.132	0.283	0.128
		192	0.181	0.337	0.181	0.337	0.176	0.331	0.178	0.334	0.177
		336	0.217	0.374	0.207	0.365	0.220	0.376	0.214	0.373	0.212
		720	0.237	0.389	0.237	0.389	0.233	0.386	0.232	0.385	0.244
	promotion		4.275%	0.077%	13.319%	5.711%	21.947%	14.023%	24.083%	15.655%	92.844%
			71.394%								
ETTm1	original	96	0.033	0.140	0.056	0.183	0.109	0.277	0.049	0.171	0.296
		192	0.058	0.186	0.081	0.216	0.151	0.310	0.157	0.317	0.429
		336	0.084	0.231	0.076	0.218	0.427	0.591	0.289	0.459	0.585
		720	0.102	0.250	0.110	0.267	0.438	0.586	0.430	0.579	0.782
	+LDM	96	0.027	0.124	0.027	0.124	0.028	0.126	0.027	0.124	0.027
		192	0.040	0.152	0.040	0.152	0.041	0.154	0.041	0.152	0.040
		336	0.053	0.175	0.053	0.175	0.055	0.177	0.055	0.176	0.054
		720	0.072	0.204	0.072	0.205	0.073	0.206	0.075	0.208	0.074
	promotion		29.397%	18.271%	41.741%	26.235%	79.413%	59.872%	70.820%	51.293%	90.704%
			68.719%								
ETTm2	original	96	0.067	0.198	0.065	0.189	0.088	0.225	0.075	0.208	0.076
		192	0.102	0.245	0.118	0.256	0.132	0.283	0.129	0.275	0.132
		336	0.130	0.279	0.154	0.305	0.180	0.336	0.154	0.302	0.160
		720	0.178	0.325	0.182	0.335	0.300	0.435	0.160	0.321	0.168
	+LDM	96	0.063	0.184	0.066	0.191	0.066	0.188	0.067	0.188	0.064
		192	0.094	0.231	0.095	0.232	0.096	0.233	0.096	0.233	0.098
		336	0.121	0.264	0.125	0.272	0.126	0.269	0.124	0.267	0.125
		720	0.175	0.323	0.176	0.327	0.179	0.329	0.176	0.327	0.176
	promotion		5.481%	4.626%	9.840%	5.406%	30.704%	19.692%	11.555%	8.727%	14.627%
			12.144%								

Generally Applicable. the predictors in the framework (Fig.4) can be replaced with any model, providing a flexible approach to enhance model performance. Table 5 shows the results of applying the framework to the FEDformer[17], Autoformer[16], Informer[28], LogTrans[57], Reformer[41] models. LDM has led to a remarkable improvement in performance, nearly rivaling the state-of-the-art (SoTA) methods. This is attributed to the fact that, on the one hand, Logspase makes the inputs to each predictor relatively short, so that all these predictors work properly. On the other hand, Multiscale decomposition decouples patterns at different scales, converting large-scale forecasting into a short-term forecasting problem, which makes effective long- term forecasting simple. Fig.6 shows long-term forecasting performance boosting results on ETT dataset.

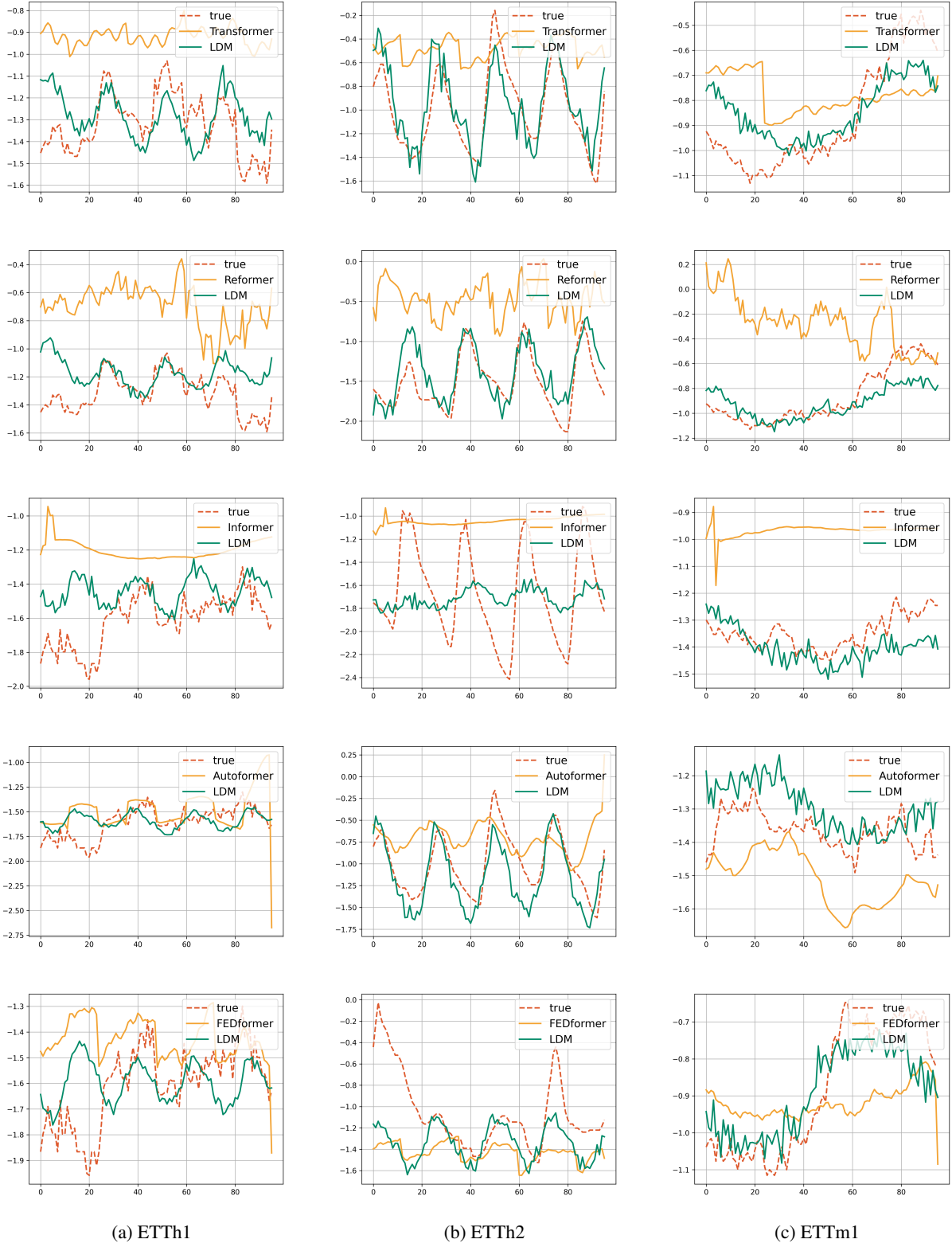


Figure 6: Results of the LDM long-term forecasting test on the ETT dataset. The image shows **the last 96 datapoints** for predicting 720 long results.

4.3 Efficiency Analysis

Previous subsection demonstrated the high performance of LDM, next we will show that LDM can also reduces cost, including training time and memory size.

Complexity. We first analyzed the computational complexity, as detailed in Table 6. As a benchmark, the computational complexity of a Transformer scales quadratically with the input length L . Crossformer and PatchTST employ patch embedding and channel independence[22], resulting in a complexity that is quadratic with respect to the number of tokens $L_{seg} \ll L$ and linear with variate number M , where p is the token size and s is the stride. The LDM is broadly categorized under this approach, and its performance largely hinges on the input length of the initial scale, L_1 . However, due to the application of Log-sparse Truncation, $\tilde{L}_1$ is significantly less than L , resulting in a substantially reduced overhead for the LDM compared to models like the Crossformer or PatchTST. This reduction enable the framework to take and use long contexts as input.

Efficiency. We conducted experiments to measure the training time and memory usage (as reported by nvidia-smi) as we increased the sequence length from 96 to 1440 for various models: PatchTST (a patch embedding Transformer) [22], FEDformer (a vanilla embedding Transformer) [17], TsMixer (a MLP model) [38], and DLinear (a linear model) [61]. These models are representative of their respective types. Utilizing the ETTm2 dataset with a fixed prediction length of 96, the results are depicted in Fig. 7. The LDM model demonstrates lower training time and memory costs compared to other Transformer-based approaches, and it performs approximately on par with Linear and MLP models. This suggests that even with a tenfold increase in input sizes and multiple levels of decomposition, LDM remains a more practical approach.

Table 6: Complexity

Model	Complexity
Transformer	$O(L^2)$
Informer	$O(L \log L)$
Autoformer	$O(L \log L)$
FEDformer	$O(L)$
Crossformer	$O\left(M \left(\frac{L}{L_{seg}}\right)^2\right)$
PatchTST	$O\left(M \left(\left\lfloor \frac{L-p}{s} \right\rfloor + 2\right)^2\right)$
Ours	$O\left(M \left(\frac{\tilde{L}_1}{L_{seg}}\right)^2\right)$

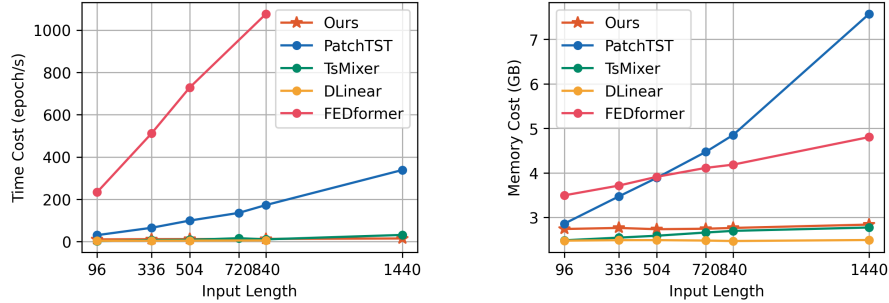


Figure 7: Time cost and memory cost (ETTM2-96).

4.4 Model Analysis

Hyper-parameter Sensitivity. We evaluate the effect of two hyper-parameters: input length L , Sparsity η and number of scales. Table 7 shows the results of L and η in different settings. L in most cases tend to favour the longer the better, but not absolutely. The situation of η is slightly more complicated, but $\frac{1}{16}$ is a good balance, which is often used in the main experiment. Decomposition makes each element simple, but larger number of scales is not always better as too many can accumulate errors in the final aggregation. Two scales which will decompose into 3 elements is optimal, as shown in Table 8.

Ablation Study. Our approach introduces two components: the Multistate Decomposition(MD) in the LDM and the Vanilla Embedding added in the predictor. Our ablation implemented on the ETTh1 dataset in line with previous work[28, 21]. Two ablation versions are 1) without VE, 2) further without MD based on 1) which degenerate into a DSW embedding (none-overlapping Patch Embedding) Transformer[21]. The results are shown in Table 9. Removing VE impairs the model in long-term forecasting (336 and 720) because, as stated previously 3.4, prediction of large-scale components is effectively transformed into a short-term prediction task due to downsampling, and VE is more advantageous in short-term forecasting. The problem is further exacerbated by remove MD.

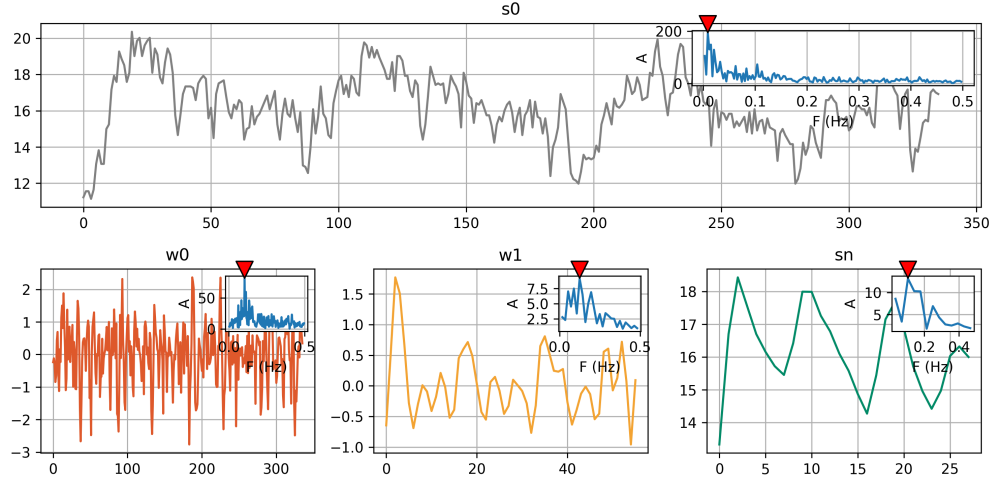


Figure 8: Multiscale Decomposition Showcase. First row shows the original signal, and second row shows the components obtained from decomposition. Frame in the upper right corner shows the signal's spectrum.

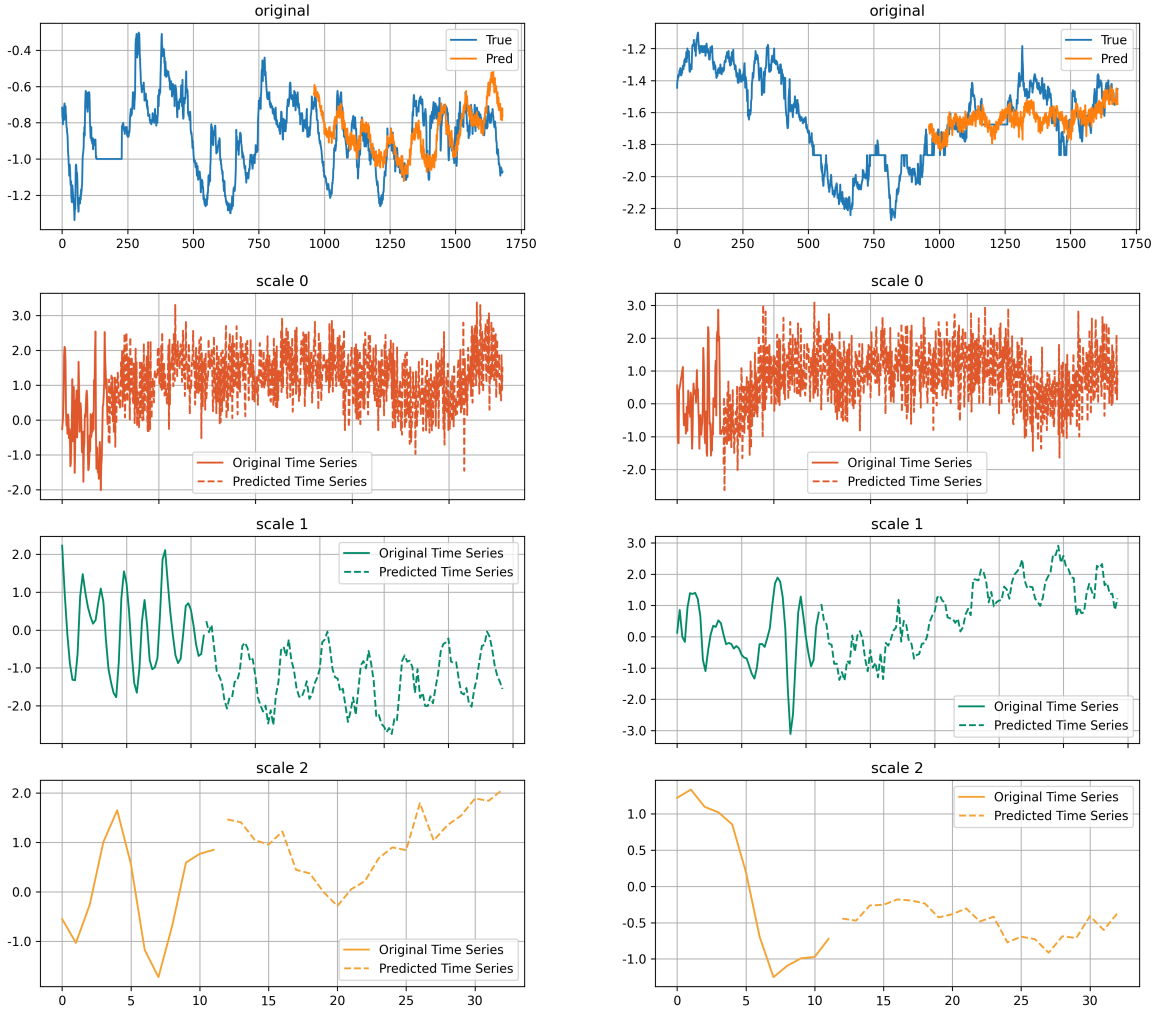


Figure 9: Expand Forecasting Results. Two example(each column) of the long-term forecasting results on ETTm1 dataset with prediction length of 720. First row shows the original sequence and the predicted results, while the rows below show the input and predicted results for each scale.

Table 7: Sensitivity analysis of input length L and sparsity η .

Dataset	ETTh2-96		ETTh2-96		ETTh2-720		ETTh2-720	
Metric	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE
L	96	0.296	0.342	0.184	0.184	0.415	0.433	0.397
	336	0.283	0.341	0.171	0.258	0.378	0.419	0.374
	960	0.283	0.339	0.165	0.254	0.416	0.448	0.337
	1680	0.279	0.34	0.164	0.252	0.417	0.456	0.331
η	1/4	0.274	0.347	0.167	0.259	0.403	0.449	0.343
	1/8	0.272	0.342	0.170	0.266	0.411	0.452	0.345
	1/16	0.276	0.347	0.164	0.260	0.417	0.456	0.331
	1/32	0.278	0.350	0.175	0.268	0.410	0.451	0.330

Table 8: Analysis on Number of Scales on ETTh1 dataset. We use four different numbers of scale sets from 1 to 4.

Scale set	[24]		[24, 168]		[12, 24, 168]		[6, 12, 24, 168]	
Metric	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE
96	0.370	0.397	0.366	0.395	0.380	0.404	0.376	0.400
192	0.409	0.419	0.401	0.415	0.413	0.420	0.413	0.420
336	0.424	0.431	0.418	0.426	0.431	0.432	0.434	0.432
720	0.440	0.461	0.427	0.449	0.434	0.454	0.441	0.458

Table 9: Ablation results on ETTh1 dataset. VE means Vanilla Embedding, and MD means Multiscale Decomposition.

Models	VE+MD		w/o VE		w/o MD	
Metric	MSE	MAE	MSE	MAE	MSE	MAE
96	0.366	0.395	0.377	0.407	0.38	0.407
192	0.401	0.415	0.411	0.427	0.413	0.429
336	0.418	0.426	0.441	0.451	0.431	0.449
720	0.427	0.449	0.452	0.471	0.483	0.487

4.5 Forecastability and Interpretability

Forecastability. Previous work[60] has suggested that the forecastability of a sequence is related to its information entropy, with lower entropy indicating stronger predictability. A sparse spectrum, where the energy of the signal is concentrated in a few main frequencies, ensures lower information entropy. As shown in the small frames in Fig.8, multi-scale decomposition results in sparse spectra at each scale, with most of the energy concentrated around a primary frequency component. This decomposition not only simplifies the data spectrum but also makes underlying trends and cycles more apparent. Consequently, this helps models make more accurate predictions by reducing data complexity.

Interpretability. The interpretability of our method stems from the separation of features at different scales. This separation makes the features at each scale more distinct and independent, which facilitates identification and interpretation. Fig.9 shows examples of expand forecasting results. By observing the model’s behavior at different scales facilitates a better understanding of the model. A simple example of using it as a debugging method is to check whether the predicted results of each component meet expectations.

5 Conclusion

In this paper, we explore multiscale analysis methods for LTSF and introduce the Logsparse Decomposable Multiscaling (LDM) framework. Our findings indicate that the decomposition-based multiscale analysis method outperforms traditional downsampling methods in terms of performance, versatility, and efficiency, even the efficiency of single-scale models. We further show that multiscale decomposition effectively reduces information entropy by generating a sparse

spectrum, thereby clarifying and distinguishing the features at each scale. This improvement in feature distinctiveness and independence significantly enhances the model’s efficiency and predictability. Additionally, by integrating the Logsparse scale, we minimize model overfitting noise and enhance model’s ability to manage longer contexts. We hope that our work offers insights for advancing LTSF.

Acknowledgments

This work was supported by National Natural Science Foundation, China (No. 62172123), the Key Research and Development Program of Heilongjiang (Grant No. 2022ZX01A36), the Special projects for the central government to guide the development of local science and technology, China (No. ZY20B11), the Harbin Manufacturing Technology Innovation Talent Project (No. CXRC20221104236), Natural Science Foundation of Heilongjiang Province of China (Grant No. YQ2021F007).

References

- [1] O. B. Sezer, M. U. Gudelek, A. M. Ozbayoglu, Financial time series forecasting with deep learning: A systematic literature review: 2005–2019, *Applied soft computing* 90 (2020) 106181.
- [2] M. Samonas, *Financial forecasting, analysis, and modelling: a framework for long-term forecasting*, John Wiley & Sons, 2015.
- [3] O. B. Sezer, M. U. Gudelek, A. M. Ozbayoglu, Financial time series forecasting with deep learning: A systematic literature review: 2005–2019, *Applied soft computing* 90 (2020) 106181.
- [4] R. Bala, R. P. Singh, et al., Financial and non-stationary time series forecasting using lstm recurrent neural network for short and long horizon, in: *2019 10th international conference on computing, communication and networking technologies (ICCCNT)*, IEEE, 2019, pp. 1–7.
- [5] R. J. Hyndman, S. Fan, Density forecasting for long-term peak electricity demand, *IEEE Transactions on Power Systems* 25 (2) (2009) 1142–1153.
- [6] C. Ying, W. Wang, J. Yu, Q. Li, D. Yu, J. Liu, Deep learning for renewable energy forecasting: A taxonomy, and systematic literature review, *Journal of Cleaner Production* 384 (2023) 135414.
- [7] I. Ghalekhondabi, E. Ardjmand, G. R. Weckman, W. A. Young, An overview of energy demand forecasting methods published in 2005–2015, *Energy Systems* 8 (2017) 411–447.
- [8] E. I. Vlahogianni, M. G. Karlaftis, J. C. Golias, Short-term traffic forecasting: Where we are and where we’re going, *Transportation Research Part C: Emerging Technologies* 43 (2014) 3–19.
- [9] C. Zhang, P. Patras, Long-term mobile traffic forecasting using deep spatio-temporal neural networks, in: *Proceedings of the Eighteenth ACM International Symposium on Mobile Ad Hoc Networking and Computing*, 2018, pp. 231–240.
- [10] L. Qu, W. Li, W. Li, D. Ma, Y. Wang, Daily long-term traffic flow forecasting based on a deep neural network, *Expert Systems with applications* 121 (2019) 304–312.
- [11] U. K. Rout, A. Voβ, A. Singh, U. Fahl, M. Blesl, B. P. Ó. Gallachóir, Energy and emissions forecast of china over a long-time horizon, *Energy* 36 (1) (2011) 1–11.
- [12] H. Liu, G. Yan, Z. Duan, C. Chen, Intelligent modeling strategies for forecasting air quality time series: A review, *Applied Soft Computing* 102 (2021) 106957.
- [13] P. W. Kalgren, C. S. Byington, M. J. Roemer, M. J. Watson, Defining phm, a lexical evolution of maintenance and logistics, in: *2006 IEEE autotestcon*, IEEE, 2006, pp. 353–358.
- [14] H. Meng, Y.-F. Li, A review on prognostics and health management (phm) methods of lithium-ion batteries, *Renewable and Sustainable Energy Reviews* 116 (2019) 109405.
- [15] E. Zio, Prognostics and health management (phm): Where are we and where do we (need to) go in theory and practice, *Reliability Engineering & System Safety* 218 (2022) 108119.
- [16] H. Wu, J. Xu, J. Wang, M. Long, Autoformer: Decomposition transformers with auto-correlation for long-term series forecasting, *Advances in Neural Information Processing Systems* 34 (2021) 22419–22430.
- [17] T. Zhou, Z. Ma, Q. Wen, X. Wang, L. Sun, R. Jin, Fedformer: Frequency enhanced decomposed transformer for long-term series forecasting, in: *International Conference on Machine Learning*, PMLR, 2022, pp. 27268–27286.

- [18] H. Wu, T. Hu, Y. Liu, H. Zhou, J. Wang, M. Long, Timesnet: Temporal 2d-variation modeling for general time series analysis, in: The eleventh international conference on learning representations, 2022.
- [19] S. Wang, H. Wu, X. Shi, T. Hu, H. Luo, L. Ma, J. Y. Zhang, J. ZHOU, Timemixer: Decomposable multiscale mixing for time series forecasting, in: The Twelfth International Conference on Learning Representations, 2023.
- [20] C. Challu, K. G. Olivares, B. N. Oreshkin, F. G. Ramirez, M. M. Canseco, A. Dubrawski, Nhits: Neural hierarchical interpolation for time series forecasting, in: Proceedings of the AAAI Conference on Artificial Intelligence, Vol. 37, 2023, pp. 6989–6997.
- [21] Y. Zhang, J. Yan, Crossformer: Transformer utilizing cross-dimension dependency for multivariate time series forecasting, in: The eleventh international conference on learning representations, 2022.
- [22] Y. Nie, N. H. Nguyen, P. Sinthong, J. Kalagnanam, A time series is worth 64 words: Long-term forecasting with transformers, arXiv preprint arXiv:2211.14730 (2022).
- [23] E. Ogasawara, L. C. Martinez, D. De Oliveira, G. Zimbrão, G. L. Pappa, M. Mattoso, Adaptive normalization: A novel data normalization approach for non-stationary time series, in: The 2010 International Joint Conference on Neural Networks (IJCNN), IEEE, 2010, pp. 1–8.
- [24] Y. Liu, H. Wu, J. Wang, M. Long, Non-stationary transformers: Exploring the stationarity in time series forecasting, Advances in Neural Information Processing Systems 35 (2022) 9881–9893.
- [25] T. Kim, J. Kim, Y. Tae, C. Park, J.-H. Choi, J. Choo, Reversible instance normalization for accurate time-series forecasting against distribution shift, in: International Conference on Learning Representations, 2021.
- [26] E. Weinan, Principles of multiscale modeling, Cambridge University Press, 2011.
- [27] Q. Wen, T. Zhou, C. Zhang, W. Chen, Z. Ma, J. Yan, L. Sun, Transformers in time series: A survey, arXiv preprint arXiv:2202.07125 (2022).
- [28] H. Zhou, S. Zhang, J. Peng, S. Zhang, J. Li, H. Xiong, W. Zhang, Informer: Beyond efficient transformer for long sequence time-series forecasting, in: Proceedings of the AAAI conference on artificial intelligence, Vol. 35, 2021, pp. 11106–11115.
- [29] T. Zhou, Z. Ma, Q. Wen, L. Sun, T. Yao, W. Yin, R. Jin, et al., Film: Frequency improved legendre memory model for long-term time series forecasting, Advances in neural information processing systems 35 (2022) 12677–12690.
- [30] Y. Liu, T. Hu, H. Zhang, H. Wu, S. Wang, L. Ma, M. Long, itransformer: Inverted transformers are effective for time series forecasting, arXiv preprint arXiv:2310.06625 (2023).
- [31] H. Fan, B. Xiong, K. Mangalam, Y. Li, Z. Yan, J. Malik, C. Feichtenhofer, Multiscale vision transformers, in: Proceedings of the IEEE/CVF international conference on computer vision, 2021, pp. 6824–6835.
- [32] C.-Y. Wu, Y. Li, K. Mangalam, H. Fan, B. Xiong, J. Malik, C. Feichtenhofer, Memvit: Memory-augmented multi-scale vision transformer for efficient long-term video recognition, in: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition, 2022, pp. 13587–13597.
- [33] J. Gu, H. Kwon, D. Wang, W. Ye, M. Li, Y.-H. Chen, L. Lai, V. Chandra, D. Z. Pan, Multi-scale high-resolution vision transformer for semantic segmentation, in: Proceedings of the IEEE/CVF conference on computer vision and pattern recognition, 2022, pp. 12094–12103.
- [34] C.-F. R. Chen, Q. Fan, R. Panda, Crossvit: Cross-attention multi-scale vision transformer for image classification, in: Proceedings of the IEEE/CVF international conference on computer vision, 2021, pp. 357–366.
- [35] S. Siarni-Namini, N. Tavakoli, A. S. Namin, A comparison of arima and lstm in forecasting time series, in: 2018 17th IEEE international conference on machine learning and applications (ICMLA), IEEE, 2018, pp. 1394–1401.
- [36] A. Sagheer, M. Kotb, Time series forecasting of petroleum production using deep lstm recurrent networks, Neurocomputing 323 (2019) 203–213.
- [37] B. N. Oreshkin, D. Carpo, N. Chapados, Y. Bengio, N-beats: Neural basis expansion analysis for interpretable time series forecasting, arXiv preprint arXiv:1905.10437 (2019).
- [38] V. Ekambaram, A. Jati, N. Nguyen, P. Sinthong, J. Kalagnanam, Tsmixer: Lightweight mlp-mixer model for multivariate time series forecasting, in: Proceedings of the 29th ACM SIGKDD Conference on Knowledge Discovery and Data Mining, 2023, pp. 459–469.
- [39] Q. Wen, T. Zhou, C. Zhang, W. Chen, Z. Ma, J. Yan, L. Sun, Transformers in time series: A survey, arXiv preprint arXiv:2202.07125 (2022).
- [40] S. Li, X. Jin, Y. Xuan, X. Zhou, W. Chen, Y.-X. Wang, X. Yan, Enhancing the locality and breaking the memory bottleneck of transformer on time series forecasting, Advances in neural information processing systems 32 (2019).

- [41] N. Kitaev, Ł. Kaiser, A. Levskaya, Reformer: The efficient transformer, arXiv preprint arXiv:2001.04451 (2020).
- [42] J.-R. Chang, Y.-S. Chen, Pyramid stereo matching network, in: Proceedings of the IEEE conference on computer vision and pattern recognition, 2018, pp. 5410–5418.
- [43] R. Chen, S. Han, J. Xu, H. Su, Point-based multi-view stereo network, in: Proceedings of the IEEE/CVF international conference on computer vision, 2019, pp. 1538–1547.
- [44] X. Guo, K. Yang, W. Yang, X. Wang, H. Li, Group-wise correlation stereo network, in: Proceedings of the IEEE/CVF conference on computer vision and pattern recognition, 2019, pp. 3273–3282.
- [45] S. Im, H.-G. Jeon, S. Lin, I. S. Kweon, Dpsnet: End-to-end deep plane sweep stereo, arXiv preprint arXiv:1905.00538 (2019).
- [46] A. Kendall, H. Martirosyan, S. Dasgupta, P. Henry, R. Kennedy, A. Bachrach, A. Bry, End-to-end learning of geometry and context for deep stereo regression, in: Proceedings of the IEEE international conference on computer vision, 2017, pp. 66–75.
- [47] M. Xu, F. Zhan, J. Zhang, Y. Yu, X. Zhang, C. Theobalt, L. Shao, S. Lu, Wavenerf: Wavelet-based generalizable neural radiance fields, in: Proceedings of the IEEE/CVF International Conference on Computer Vision, 2023, pp. 18195–18204.
- [48] Q. Xu, W. Kong, W. Tao, M. Pollefeys, Multi-scale geometric consistency guided and planar prior assisted multi-view stereo, IEEE Transactions on Pattern Analysis and Machine Intelligence 45 (4) (2022) 4945–4963.
- [49] Z. Wei, Q. Zhu, C. Min, Y. Chen, G. Wang, Aa-rmvnet: Adaptive aggregation recurrent multi-view stereo network, in: Proceedings of the IEEE/CVF international conference on computer vision, 2021, pp. 6187–6196.
- [50] X. Gu, Z. Fan, S. Zhu, Z. Dai, F. Tan, P. Tan, Cascade cost volume for high-resolution multi-view stereo and stereo matching, in: Proceedings of the IEEE/CVF conference on computer vision and pattern recognition, 2020, pp. 2495–2504.
- [51] Y. Lee, J. Kim, J. Willette, S. J. Hwang, Mpvit: Multi-path vision transformer for dense prediction, in: Proceedings of the IEEE/CVF conference on computer vision and pattern recognition, 2022, pp. 7287–7296.
- [52] N. Takahashi, Y. Mitsufuji, Densely connected multi-dilated convolutional networks for dense prediction tasks, in: Proceedings of the IEEE/CVF conference on computer vision and pattern recognition, 2021, pp. 993–1002.
- [53] C. S. Won, Multi-scale cnn for fine-grained image recognition, IEEE Access 8 (2020) 116663–116674.
- [54] Y. Li, C.-Y. Wu, H. Fan, K. Mangalam, B. Xiong, J. Malik, C. Feichtenhofer, Mvitv2: Improved multiscale vision transformers for classification and detection, in: Proceedings of the IEEE/CVF conference on computer vision and pattern recognition, 2022, pp. 4804–4814.
- [55] A. Gasparin, S. Lukovic, C. Alippi, Deep learning for time series forecasting: The electric load case, CAAI Transactions on Intelligence Technology 7 (1) (2022) 1–25.
- [56] S. J. Brown, W. N. Goetzmann, A. Kumar, The dow theory: William peter hamilton’s track record reconsidered, The Journal of finance 53 (4) (1998) 1311–1333.
- [57] S. Li, X. Jin, Y. Xuan, X. Zhou, W. Chen, Y.-X. Wang, X. Yan, Enhancing the locality and breaking the memory bottleneck of transformer on time series forecasting, Advances in neural information processing systems 32 (2019).
- [58] M. J. Shensa, et al., The discrete wavelet transform: wedding the a trous and mallat algorithms, IEEE Transactions on signal processing 40 (10) (1992) 2464–2482.
- [59] M. Liu, A. Zeng, M. Chen, Z. Xu, Q. Lai, L. Ma, Q. Xu, Scinet: Time series modeling and forecasting with sample convolution and interaction, Advances in Neural Information Processing Systems 35 (2022) 5816–5828.
- [60] G. Goerg, Forecastable component analysis, in: International conference on machine learning, PMLR, 2013, pp. 64–72.
- [61] A. Zeng, M. Chen, L. Zhang, Q. Xu, Are transformers effective for time series forecasting?, in: Proceedings of the AAAI conference on artificial intelligence, Vol. 37, 2023, pp. 11121–11128.